App Dev Project Report

1. Student Details

Name: Ashutosh Kumar
Roll Number: 24f2003880
Email: 24f2003880@ds.study.iitm.ac.in
About Me: I am a student of the IIT Madras BS Degree Programme in Data Science and Applications, with an interest in web application development, data analytics, and building practical software applications.

---

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

To design and build a web-based application that allows users to **browse and book trekking activities**, while enabling administrators to manage treks, users, staff and bookings. Assigned staff members can manage their treks and view registered participants.

Approach:

The application was developed using a **Flask REST API backend and Vue.js frontend**. The backend handles authentication, database operations, trek management, booking management, staff assignment and role-based access control. The frontend communicates with the backend through APIs using Axios.

SQLite is used for persistent data storage. **JWT** is used for authentication and authorization. **Redis** is used for caching, while **Celery** is used for asynchronous tasks such as daily reminders, monthly reports and booking-history export.

---

3. AI/LLM Declaration

I used ChatGPT (AI/LLM) to assist in understanding concepts, troubleshooting errors, and providing coding guidance during the development of the project.

The extent of AI/LLM usage is around 25%

All final implementation, testing, debugging, and integration were performed by me**.**

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Backend REST API development
Flask-SQLAlchemy	ORM and database operations
Flask-JWT-Extended	JWT-based authentication and authorization
SQLite	Persistent database storage
Vue.js	Frontend user interface
Vue Router	Frontend routing and navigation
Axios	Communication between frontend and backend APIs
Bootstrap	User interface styling and responsive layout
Redis	Caching and support for background task processing
Celery	Asynchronous/background task processing
MailHog	Local testing of email notifications

---

5. Database Schema / ER Diagram

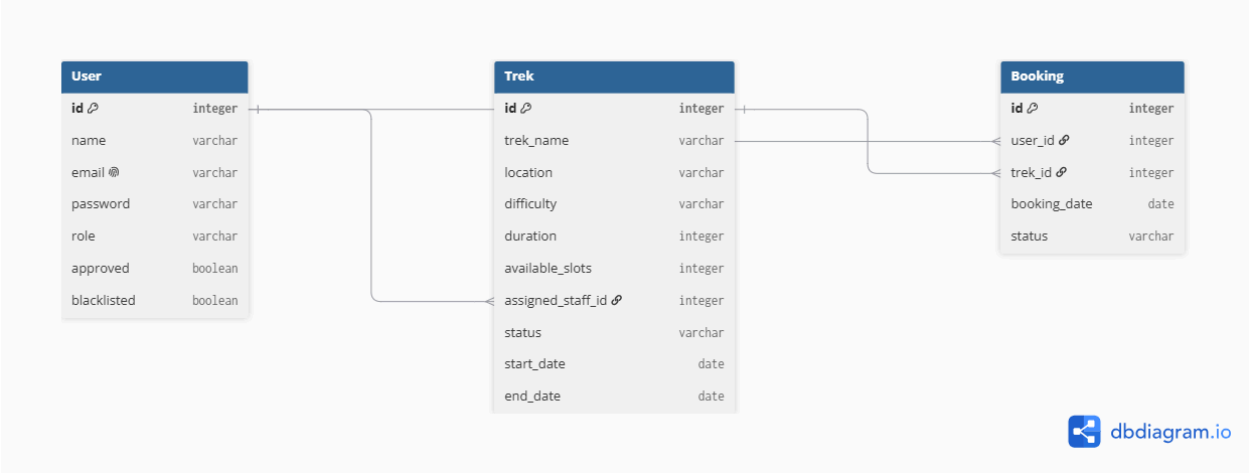
Tables:

1. **User** — stores user details, authentication information, role and account status.
2. **Trek** — stores trek name, location, difficulty, duration, available slots, dates, status and assigned staff.
3. **Booking** — stores user bookings, trek reference, booking date and booking status.

Relationships:

- One-to-Many → `User → Booking`
- One-to-Many → `Trek → Booking`
- Staff assignment → `User → Trek` through `assigned_staff_id`

ER Diagram:



---

6. API Resource Endpoints

Endpoint	Method	Description
`/register`	POST	Register a new user
`/login`	POST	Authenticate user and generate JWT
`/treks`	GET	Fetch trek information
Trek creation endpoint	POST	Create a new trek
Trek update endpoint	PUT	Update trek information
Trek deletion endpoint	DELETE	Delete a trek
Booking endpoint	POST	Create a trek booking
Booking endpoint	GET	Retrieve booking information
Staff/Admin endpoints	GET/POST/PUT/DELETE	Manage staff and administrative operations

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features

Architecture Overview:

- **`backend/app.py`** – main Flask application entry point and API configuration
- **`backend/models.py`** – SQLAlchemy database models
- **`backend/config.py`** – application configuration
- **`backend/mail_service.py`** – email sending functionality
- **`backend/tasks.py`** – Celery background tasks
- **`backend/celery_worker.py`** – Celery worker configuration/execution
- **`frontend/`** – Vue.js frontend application
- **Redis** – caching and background task infrastructure
- **SQLite** – application database

Implemented Features:

- User registration and login
- JWT-based authentication
- Role-based access for Admin, Staff and Users
- Trek creation, update and deletion
- Trek search and filtering
- Trek booking
- Duplicate booking prevention
- Available-slot and booking validation
- Staff assignment to treks
- Staff management of assigned treks
- Participant list for assigned staff
- Booking status management
- Daily trek reminder emails
- Monthly report generation
- Booking history CSV export
- Redis caching
- Celery asynchronous/background tasks

Additional Features:

- Trek filtering by name, location, difficulty and duration
- Redis caching for frequently accessed trek data

- Local email notification testing using MailHog
- Asynchronous CSV export using Celery

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1whflkNkEHmdxcPwli8QiyB2yBSwqFWrN/view?usp=sharin